
Actividad Práctica: "Creando un Supercomputador Casero" 💻

Esta actividad tiene como objetivo que los estudiantes experimenten de forma práctica los conceptos de multicomputadores, redes de interconexión y computación paralela utilizando 8 computadoras Pentium IV. La duración estimada es de 2 horas.

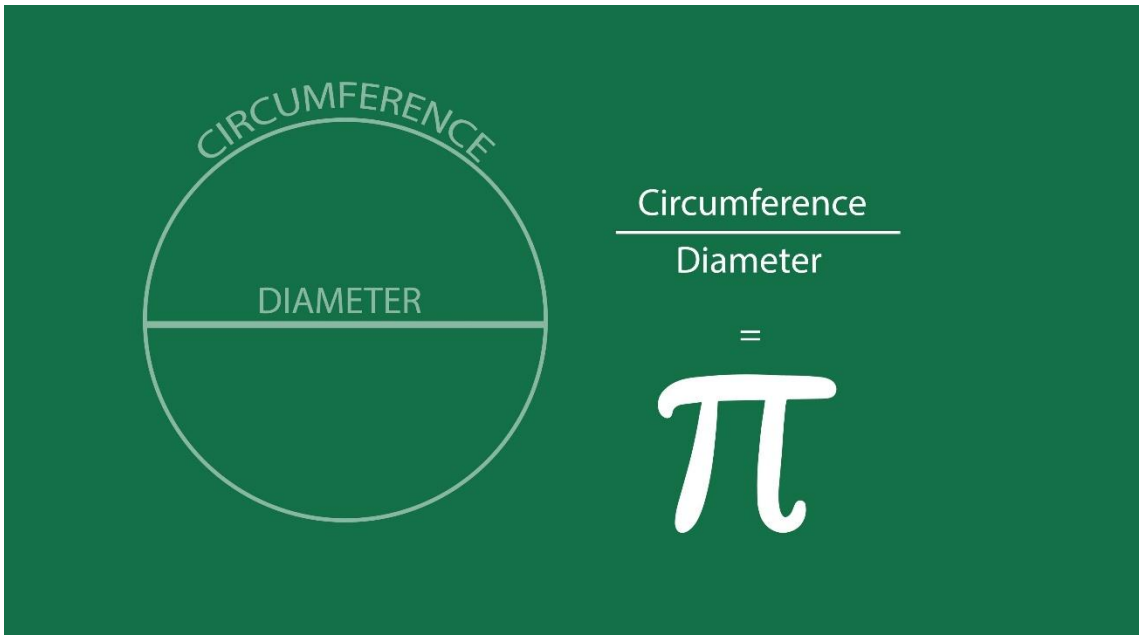
Fase 1: Preparación del "Cluster" (30 minutos)

- **Objetivo: Comprender la arquitectura de un multicomputador a nivel de hardware y redes.**
- **Materiales: 8 computadoras Pentium IV, un switch de red de 8 puertos o más, cables de red (RJ-45), unidades USB con el software necesario.**
- **Procedimiento:**
    1. **Conexión Física: Guía a los estudiantes para que conecten las 8 computadoras al interruptor de red. Explícales que cada PC es un nodo (un "computador" en el multicomputador) y que el switch y los cables forman la red de interconexión .**
    2. **Configuración de la Red: Asigne a cada equipo una dirección IP estática en el mismo rango de red (ej. 192.168.1.10 a 192.168.1.17). Explica que esta configuración es el "lenguaje" para que se comuniquen entre sí, similar a la comunicación por paso de mensajes que se discutió en teoría.**
    3. **Verificación de la Conectividad: Pide a los estudiantes que usen el comando pingde un nodo a otro para verificar que todos los computadores están conectados correctamente.**

Fase 2: El Desafío de la Computación Paralela (60 minutos)

- **Objetivo: Ejecutar un programa en paralelo para entender el concepto de aceleración y eficiencia.**
- **Materiales:**
    - **Software de Computación Paralela: Instale en los 8 equipos un software de computación paralela distribuida, como MPI (Message Passing Interface) . Un paquete como MPICH o Open MPI es ideal para este propósito.**

- **Programa de Prueba: Prepare un programa simple escrito en C o Python que pueda ser paralelo. Un ejemplo clásico es el cálculo del número Pi.**



Con licencia de Google

o la suma de una matriz muy grande. El programa debe tener una parte secuencial (lectura de datos) y una parte paralela (el cálculo en sí).

- **Procedimiento:**

  1. **Explicación del Problema: Explica que el programa divide el trabajo en 4 partes, y cada computadora se encargará de una porción del cálculo. Esto ilustra el concepto de MIMD (Múltiples Instrucciones, Múltiples Datos).**
  2. **Ejecución Secuencial: Primero, ejecuta el programa en una sola computadora. Cronómetro el tiempo de ejecución y regístralo comoT1.**
  3. **Ejecución Paralela: Luego, ejecuta el programa en los 4 computadores usando MPI. Muestra a los estudiantes cómo el programa se distribuye y cómo cada nodo reporta su resultado a un nodo maestro. Cronómetro el tiempo de ejecución y regístralo como T4.**
  4. **Análisis de resultados:**
     - **Cálculo de Aceleración: Pídeles que calculan la aceleración (S) usando la fórmulaS (N)=T1/ Tnorte.**
     - **Cálculo de Eficiencia: Pídeles que calculan la eficiencia (mi) usando la fórmulaE (N)=S (N) / N.**

- **Discusión Guiada: "¿Por qué el valor de la aceleración no fue 8, y la eficiencia no fue 1?" Esto los llevará a discutir de forma natural los desafíos del paralelismo, como la parte secuencial del código (Ley de Amdahl), la latencia de la red y la sobrecarga de la comunicación.**

Fase 3: Reflexión y Conceptualización (30 minutos)

- **Objetivo: Conectar la experiencia práctica con los conceptos teóricos del curso.**
- **Actividad: Pide a los estudiantes que, en un pequeño grupo de discusión, respondan las siguientes preguntas:**
  - **¿Cómo se relaciona la arquitectura multicomputadora que construyeron con la definición teórica?**
  - **¿Qué papel jugó el switch de red en el rendimiento del programa? ¿Qué topología de red indirecta implementaron?**
  - **¿Dónde se vieron reflejados los conceptos de paso de mensajes y memoria distribuida en esta actividad?**
  - **¿Por qué creen que el programa no tuvo una aceleración perfecta? ¿Qué factores de los vistos en clase (latencia de red, parte secuencial) influyeron en el resultado?**

Esta actividad convierte la teoría abstracta en una experiencia tangible, permitiendo que los estudiantes no solo recuerden las definiciones, sino que comprendan su impacto en el mundo real.

Guía de Laboratorio: Creación de un Miniclúster Multicomputador (Fase 1)

Materia: Arquitectura de los Computadores II

Unidad: II. Multicomputadores

Duración: 30 minutos

Objetivo: Comprender la arquitectura de un multicomputador, desde su hardware hasta su configuración de red, para habilitar la comunicación entre nodos.

Materiales por grupo:

- **4** computadoras Pentium IV 🖥️
- **1** switch de red (al menos 4 puertos)
- **4** cables de red RJ-45
- **1** unidad USB con software de verificación (opcional, aunque se puede usar el cmdde Windows)
- **1** ficha de registro de datos (sección 3 de esta guía)

---

1. Introducción al Laboratorio

¡Bienvenidos a la primera fase de nuestro proyecto de clúster! En esta sesión, transformarán cuatro computadoras individuales en un sistema multicomputador funcional.

Recuerden que un **multicomputador** es una arquitectura paralela donde cada procesador tiene su propia memoria privada, y la comunicación entre ellos se realiza mediante el intercambio de mensajes a través de una red de interconexión.

En este laboratorio, cada una de sus cuatro computadoras actuará como un **nodo** independiente, y el switch junto con los cables será nuestra **red de interconexión** .

---

2. Procedimiento Guiado

Siga estos pasos con atención para configurar su miniclúster.

Paso 2.1: Conexión Física de la Red

- Tomen el switch de red y colóquenlo en el centro de su área de trabajo.
- Conecte el extremo de un cable RJ-45 al puerto rojo (tarjeta Ethernet) de la primera computadora.

- Conecte el otro extremo del mismo cable a uno de los puertos del switch.
- Repitan el proceso para las otras tres computadoras, conectándolas individualmente al switch. Asegúrese de que los cables estén bien ajustados.

¡Verificación! Observe las luces indicadoras en el interruptor para cada puerto utilizado. Una luz verde o ámbar encendida indica que la conexión física es correcta.

Paso 2.2: Configuración de la Red Lógica

La conexión física no es suficiente; las computadoras necesitan un "lenguaje" para hablar entre sí. Este lenguaje será la configuración de las **direcciones IP estáticas** .

- En cada una de las cuatro computadoras, abra la ventana de **Configuración de Red** .
    - En Windows: Vayan a Panel de control > Redes e Internet > Centro de redes y recursos compartidos > Cambiar configuración del adaptador. Haga clic derecho sobre la conexión Ethernet y seleccione Propiedades.
- Busquen la opción Protocolo de Internet versión 4 (TCP/IPv4)y hagan clic en Propiedades.
- Seleccionen la opción **Usar la siguiente dirección IP**.
- Para cada computadora, asigna la siguiente información de red, registrando los datos en la tabla de la sección 3:

Computadora (Nodo)	Dirección IP	Máscara de subred	Puerta de enlace predeterminada
Nodo 1	192.168.1.10	255.255.255.0	192.168.1.1
Nodo 2	192.168.1.11	255.255.255.0	192.168.1.1
Nodo 3	192.168.1.12	255.255.255.0	192.168.1.1
Nodo 4	192.168.1.13	255.255.255.0	192.168.1.1

- Haga clic en **Aceptar** y cierren las ventanas.

Paso 2.3: Verificación de la Conectividad (Comunicación por Mensajes)

Ahora, verifiquen si los nudos pueden "hablar" entre sí.

- En la primera computadora (**Nodo 1**), abran el Símbolo del sistema(cmd).
- Escriban el siguiente comando y presionen Enter para enviar un "mensaje" (un paquete ping) al **Nodo 2** :

    - ping 192.168.1.11

- Verifiquen que la respuesta sea exitosa, con un "Tiempo de respuesta" bajo y 0% de pérdida de paquetes.
- Ahora, repitan el ping desde el **Nodo 1** a los otros dos nodos: ping 192.168.1.12y ping 192.168.1.13.
- Finalmente, realicen la misma prueba desde cualquier otro nodo (ej. **Nodo 3**) para asegurarse de que la conectividad es **bidireccional** .

---

3. Registro de Datos y Análisis

Complete la siguiente tabla con los resultados obtenidos durante la verificación.

Desde el Nodo	Hacia el Nodo	Comando deping	¿Respuesta exitosa? (Sí/No)	Tiempo promedio (ms)
Nodo 1	Nodo 2	ping 192.168.1.11		
Nodo 1	Nodo 3	ping 192.168.1.12		
Nodo 1	Nodo 4	ping 192.168.1.13		
Nodo 3	Nodo 1	ping 192.168.1.10		

Preguntas de reflexión:

1. En la arquitectura que acaban de construir, ¿qué componentes representan a la **memoria distribuida** ?
2. ¿Qué diferencia hay entre la conexión física (paso 2.1) y la configuración lógica (paso 2.2)?
3. Según lo visto en clase, ¿qué **topología de red** indirecta implementaron con el switch y los cables? ¿Por qué?

Al completar esta fase, su grupo habrá logrado el primer gran paso: crear un entorno de hardware y rojo listo para la computación paralela. ¡En la siguiente sesión, daremos vida al clúster ejecutando un programa paralelo!

de Laboratorio: El Desafío de la Computación Paralela (Fase 2)

Materia: Arquitectura de los Computadores II

Unidad: II. Multicomputadores

Duración: 60 minutos

Objetivo: Medir el rendimiento de su miniclúster multicomputador, ejecutando un programa en modo secuencial y en modo paralelo para calcular la **aceleración** y la **eficiencia**.

Materiales por Grupo:

- **4** computadoras (nodos del clúster).
- **1** switch y cables de red (previamente conectados).
- **1** unidad USB con el programa de prueba (**cálculo de Pi**) y el software **MPI (Message Passing Interface)** instalado.
- **1** cronómetro (puede ser la función del teléfono móvil).
- **1** ficha de registro de datos (sección 3 de esta guía).

---

1. Introducción al Desafío

En la fase anterior, construyeron la infraestructura de su miniclúster. Ahora, darán vida a esa red ejecutando un programa en paralelo. El objetivo es entender cómo la división del trabajo entre los nodos impacta en el rendimiento general del sistema. Para ello, medirán el tiempo de ejecución secuencial y paralelo para calcular dos métricas clave: la **aceleración** (speedup) y la **eficiencia**.

2. Procedimiento de Medición

Sigan estos pasos para ejecutar el programa de prueba y registrar los datos.

Paso 2.1: El Problema de Prueba: Cálculo de Pi

El programa de prueba que van a usar calcula el número **Pi** (π) mediante un método numérico. Este algoritmo es ideal para la computación paralela porque el cálculo total puede dividirse en partes más pequeñas, donde cada procesador resuelve una porción del problema de forma independiente.

- **Parte secuencial:** La inicialización del programa y la suma final de los resultados parciales de cada nodo.
- **Parte paralela:** El cálculo de la porción asignada a cada procesador.

Paso 2.2: Medición del Tiempo de Ejecución Secuencial (T1)

1. En una sola computadora (**Nodo 1**), abran la carpeta del programa de prueba.
2. Abran una ventana de **Símbolo del sistema** (cmd) en esa misma carpeta.
3. **Inicien el cronómetro.**
4. Ejecuten el programa en modo secuencial. Por ejemplo, si el programa se llama pi_calc.exe, ejecuten el comando:
    - pi_calc.exe
5. **Detengan el cronómetro** tan pronto como el programa termine y muestre el resultado en la pantalla.
6. Registren el tiempo total de ejecución en su ficha de datos como **T1**.

Paso 2.3: Medición del Tiempo de Ejecución Paralelo (T4)

Ahora, ejecutarán el mismo programa, pero distribuyendo el trabajo entre los **cuatro nodos** de su clúster.

1. En una de las computadoras (el **Nodo 1**, que actuará como nodo maestro), abran una ventana de cmd.
2. **Inicien el cronómetro.**
3. Ejecuten el programa en modo paralelo usando el comando de MPI. Este comando le indica a MPI que distribuya el trabajo en los cuatro nodos del clúster:
    - mpiexec -n 4 -host 192.168.1.10,192.168.1.11,192.168.1.12,192.168.1.13 pi_calc.py

Para la ejecución del programa en modo paralelo usando **MPI (Message Passing Interface)**, es necesario realizar una instalación y configuración en las computadoras con Windows, ya que **MPI no es una característica nativa** del sistema operativo. A continuación, se detalla el proceso completo que debes incluir en la guía de laboratorio para guiar a los estudiantes de manera práctica.

Instalación y Configuración del Entorno de MPI en Windows

Antes de ejecutar el programa en paralelo, cada una de las cuatro computadoras debe tener instalado un entorno de tiempo de ejecución de MPI. El paquete más recomendado para Windows es **Microsoft MPI (MS-MPI)**, ya que es oficial y se integra bien con el sistema operativo.

Paso 1: Instalación de MS-MPI en cada Nodo

i. En cada una de las cuatro computadoras, descarga el instalador de **Microsoft MPI** desde el sitio web oficial de Microsoft. Asegúrate de obtener tanto el **SDK** como el **Redistributable Package**.

ii. Ejecuta el instalador del **Redistributable Package** (MS-MPI.exe). Esto instala los archivos necesarios para ejecutar los programas MPI.

iii. Ejecuta el instalador del **SDK** (MS-MPI SDK.msi). El SDK es necesario para compilar programas MPI, aunque para esta práctica ya tendrás el ejecutable listo.

Paso 2: Configuración de la Red y Credenciales

Para que los nodos se comuniquen entre sí, MS-MPI necesita que las computadoras puedan acceder a los recursos compartidos.

i. **Habilitar el Servicio de MPI:** En cada computadora, abre una ventana de **Símbolo del sistema** (cmd) como administrador y ejecuta el siguiente comando:

ii. smpd -install

Esto instala el servicio smpd (Process Manager) que permite a MS-MPI gestionar los procesos en los diferentes nodos.

iii. **Crear un Archivo de Host:** En el **Nodo 1**, abre el Bloc de Notas y crea un archivo llamado hosts.txt con las direcciones IP de todos los nodos. Cada dirección debe ir en una línea separada.

192.168.1.10

192.168.1.11

192.168.1.12

192.168.1.13

Guarda este archivo en la misma carpeta donde se encuentra el ejecutable del programa (pi_calc.py).

Paso 3: Ejecución del Programa en Paralelo

Con el entorno configurado, ahora puedes ejecutar el programa en paralelo.

i. Abre una ventana de **Símbolo del sistema** (cmd) en el **Nodo 1** (la computadora desde la que lanzarás el programa).

ii. **Inicia el cronómetro.**

iii. Ejecuta el programa utilizando el comando mpiexec. Este comando es la clave para la distribución de la carga de trabajo.

iv. mpiexec -n 4 -hosts hosts.txt pi_calc.py

- mpiexec: Es el comando principal de MPI para ejecutar un programa en paralelo.
- -n 4: Le indica a MPI que lance 4 instancias del programa.
- -hosts hosts.txt: Le dice a MPI que lea las direcciones IP de los nodos desde el archivo hosts.txt que creaste.
- pi_calc.exe: Es el nombre del programa ejecutable.

v. **Detén el cronómetro** tan pronto como el programa finalice y el **Nodo 1** muestre el resultado final.

Con este procedimiento, los estudiantes verán cómo se coordina la computación en los cuatro equipos a través del software MPI, permitiéndoles observar la diferencia en el tiempo de ejecución y entender la sobrecarga de la comunicación por paso de mensajes en un sistema multicomputador real.

Ver anexo 1 (codigo del programa en Python o en C)

4. **Detengan el cronómetro** cuando el programa termine y el nodo maestro haya recolectado todos los resultados.
5. Registren el tiempo total de ejecución en su ficha de datos como **T4**.

---

3. Análisis de Resultados y Preguntas de Reflexión

Completen la siguiente tabla y respondan las preguntas para analizar el rendimiento de su clúster.

Parámetro	Valor Obtenido
Tiempo de ejecución secuencial (T1)	
Tiempo de ejecución paralelo (T4)	
Número de procesadores (N)	4

Exportar a Hojas de cálculo

Cálculos:

1. **Aceleración (Speedup)**: S(N)=T1/TN

- Sustituyan los valores obtenidos: S(4)=T1/T4
- Resultado: _____

2. **Eficiencia (Efficiency)**: E(N)=S(N)/N
    - Sustituyan los valores obtenidos: E(4)=S(4)/4
    - Resultado: _____

Preguntas de Reflexión (para discusión en grupo):

1. ¿El valor de la aceleración (S) fue igual a 4? ¿Por qué creen que no se logró un **rendimiento lineal**?
2. Mencionen al menos dos factores que, basándose en la teoría vista en clase (Ley de Amdahl, latencia de la red, sobrecarga de comunicación), podrían haber afectado la eficiencia de su clúster.
3. ¿Cómo se manifestó en esta práctica el concepto de **comunicación por paso de mensajes**? ¿Qué parte del proceso dependió de esta comunicación?

Al completar esta fase, habrán evaluado de primera mano los desafíos y beneficios de la computación paralela en un sistema multicomputador.

Anexo 1 código y opciones de ejecución en Python y C

A continuación, te muestro el código fuente de un programa para calcular Pi, junto con las instrucciones para compilarlo y un resumen de cómo se distribuye la carga de trabajo entre los procesadores.

---

1. CÓDIGO DEL PROGRAMA: PI_CALC.C

Este código es un ejemplo clásico para la computación paralela con MPI. Utiliza la aproximación de Monte Carlo, que es fácil de paralelizar. La lógica es simple: cada proceso (o nodo) genera un subconjunto de puntos aleatorios y cuenta cuántos caen dentro de un círculo de radio 1. Al final, se suman todos los conteos para obtener una estimación final de Pi.

C

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <mpi.h>
```

```
int main(int argc, char* argv[]) {
    int num_nodos, rank;
    long long int i, num_puntos_totales, num_puntos_locales;
    long long int puntos_dentro_circulo_locales = 0;
    long long int puntos_dentro_circulo_totales = 0;
    double x, y, distancia;
    double pi_estimado;
    double tiempo_inicio, tiempo_fin;

    // Inicializar el entorno MPI
    MPI_Init(&argc, &argv);

    // Obtener el número total de nodos (procesos)
    MPI_Comm_size(MPI_COMM_WORLD, &num_nodos);

    // Obtener el rango (ID) de este nodo
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Definir el número total de puntos para la estimación
    // Solo el nodo principal (rank 0) lee el argumento de entrada
    if (rank == 0) {
        if (argc < 2) {
            printf("Uso: mpiexec -n <num_nodos> pi_calc <num_puntos>\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
        num_puntos_totales = atoll(argv[1]);
        printf("Calculando Pi con %lld puntos aleatorios...\n", num_puntos_totales);
```

```
    tiempo_inicio = MPI_Wtime();
  }

  // Distribuir el número total de puntos a todos los nodos
  // Esto es una parte secuencial del código
  MPI_Bcast(&num_puntos_totales, 1, MPI_LONG_LONG_INT, 0,
MPI_COMM_WORLD);

  // Cada nodo calcula su porción de puntos
  num_puntos_locales = num_puntos_totales / num_nodos;

  // Generar la semilla aleatoria para cada nodo
  srand(rank);

  // Bucle principal para generar y probar los puntos
  for (i = 0; i < num_puntos_locales; i++) {
    x = (double)rand() / RAND_MAX * 2.0 - 1.0;
    y = (double)rand() / RAND_MAX * 2.0 - 1.0;
    distancia = sqrt(x * x + y * y);
    if (distancia <= 1.0) {
      puntos_dentro_circulo_locales++;
    }
  }

  // Sumar los conteos locales en el nodo principal (rank 0)
  // Esto ilustra la comunicación por paso de mensajes
  MPI_Reduce(&puntos_dentro_circulo_locales, &puntos_dentro_circulo_totales, 1,
MPI_LONG_LONG_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

```
    // El nodo principal (rank 0) calcula el resultado final y lo imprime
    if (rank == 0) {
        pi_estimado = 4.0 * (double)puntos_dentro_circulo_totales /
(double)num_puntos_totales;
        tiempo_fin = MPI_Wtime();
        printf("Pi estimado: %f\n", pi_estimado);
        printf("Tiempo de ejecucion: %f segundos\n", tiempo_fin - tiempo_inicio);
    }

    // Finalizar el entorno MPI
    MPI_Finalize();

    return 0;
}
```

---

2. Compilación y Ejecución del Programa

Para obtener el archivo pi_calc.exe que se usa en la guía de laboratorio, primero debes compilar el código fuente.

Paso 1: Compilación

Abre el Símbolo del sistema y usa el compilador de MPI. Si usas Microsoft MPI, el compilador se llama mpicc.

- **Comando:** mpicc pi_calc.c -o pi_calc.exe
- mpicc: Es el compilador de C que viene con MPI. Se encarga de vincular el código con las bibliotecas de MPI necesarias.
- pi_calc.c: Es el nombre del archivo fuente que creaste.
- -o pi_calc.exe: Le indica al compilador que genere un archivo ejecutable llamado pi_calc.exe.

Paso 2: Ejecución

Una vez compilado, puedes ejecutar el programa de dos maneras, lo que te permitirá medir el tiempo de ejecución secuencial y paralelo.

- **Ejecución Secuencial (en un solo nodo):**
    - **Comando:** pi_calc.exe 100000000
    - 100000000: Es el número total de puntos que se usarán en el cálculo. Este es el valor de entrada para el programa.
- **Ejecución Paralela (en cuatro nodos):**
    - **Comando:** mpiexec -n 4 -hosts hosts.txt pi_calc.exe 100000000
    - mpiexec: El lanzador de procesos de MPI.
    - -n 4: Lanza 4 instancias del programa.
    - -hosts hosts.txt: Lee las direcciones IP de los nodos desde el archivo hosts.txt que creaste en la guía.
    - pi_calc.exe: El programa a ejecutar.
    - 100000000: El número total de puntos a calcular, que se distribuye automáticamente entre los 4 nodos.

Este proceso de compilación y ejecución demuestra la transición de un código simple a un programa que aprovecha la potencia de múltiples computadoras para resolver un problema de forma más rápida.

PARA PROGRAMA EN PYTHON pi_calc.py

Si usas **Python**, no es necesario compilar el código a un archivo .exe para ejecutarlo en paralelo con **MPI**. El intérprete de Python se encarga de ejecutar el código directamente, por lo que el proceso es más sencillo y no requiere un paso de compilación adicional.

Programa en Python para el Cálculo de Pi

Aquí tienes el código de ejemplo, equivalente al de C, pero escrito en Python. Para que funcione, el módulo **mpi4py** debe estar instalado en cada una de las máquinas.

Python

```
import sys

import random

import time

from mpi4py import MPI
```

```
def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    # El nodo 0 lee el número total de puntos
    if rank == 0:
        if len(sys.argv) < 2:
            print("Uso: mpiexec -n <num_nodos> python pi_calc.py <num_puntos>")
            comm.Abort(1)

        num_puntos_totales = int(sys.argv[1])
        print(f"Calculando Pi con {num_puntos_totales:,} puntos aleatorios...")
        tiempo_inicio = time.time()
    else:
        num_puntos_totales = None

    # El nodo 0 distribuye el número total de puntos a los demás nodos
    num_puntos_totales = comm.bcast(num_puntos_totales, root=0)

    # Cada nodo calcula su porción de puntos
    num_puntos_locales = num_puntos_totales // size
    puntos_dentro_circulo_locales = 0

    # Generar la semilla aleatoria para cada nodo
    random.seed(rank)

    for _ in range(num_puntos_locales):
```

```
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)
        distancia = x**2 + y**2
        if distancia <= 1.0:
            puntos_dentro_circulo_locales += 1

    # Sumar los conteos locales en el nodo 0
    puntos_dentro_circulo_totales = comm.reduce(puntos_dentro_circulo_locales,
op=MPI.SUM, root=0)

    # El nodo 0 calcula el resultado final y lo imprime
    if rank == 0:
        pi_estimado = 4.0 * puntos_dentro_circulo_totales / num_puntos_totales
        tiempo_fin = time.time()
        print(f"Pi estimado: {pi_estimado:.6f}")
        print(f"Tiempo de ejecución: {tiempo_fin - tiempo_inicio:.4f} segundos")

if __name__ == "__main__":
    main()
```

Proceso de Instalación y Ejecución

Para trabajar con Python en un clúster con MPI, debes seguir estos pasos en cada máquina:

1. **Instalar Python:** Asegúrate de que Python esté instalado en cada una de las computadoras.
2. **Instalar mpi4py:** Abre la línea de comandos y ejecuta el siguiente comando para instalar la biblioteca mpi4py, que es el enlace entre Python y la biblioteca de MPI instalada en el sistema (como MS-MPI).

Bash

```
pip install mpi4py
```

3. **Ejecución Paralela:** Una vez que el archivo pi_calc.py y el archivo hosts.txt (con las IPs de los nodos) estén en el nodo maestro, la ejecución es similar a la versión en C, pero invocando al intérprete de Python.

Comando para ejecución en paralelo:

Bash

mpiexec -n 4 -hosts hosts.txt python pi_calc.py 100000000

- **python**: Llama al intérprete de Python para ejecutar el script.
- **pi_calc.py**: El nombre del archivo del programa.
- **100000000**: El argumento que el programa utiliza para definir el número de puntos a calcular.

Este enfoque simplifica el proceso al eliminar el paso de compilación, haciendo la práctica más ágil para los estudiantes.